Aufgabe 4.6

Kritische Reflektion zu TDD, DDD und LLM-gestützte Entwicklung

Während der Entwicklung von MediTrack konnten wir praktische Erfahrungen mit Test-Driven Development (TDD) sammeln. Dabei haben wir zunächst JUnit-Tests geschrieben und anschließend die passende Implementierung entwickelt, wodurch wir frühzeitig überprüfen konnten, ob die Funktionen wie erwartet arbeiten. Besonders hilfreich waren die Tests für ungültige Namen, E-Mail-Adressen und Telefonnummern sowie für die Aktualisierung von Patientendaten, bei der vorhandene Daten trotz fehlerhafter Eingaben erhalten bleiben sollten. Eine Herausforderung bestand darin, zunächst geeignete Testfälle zu formulieren, was teilweise zusätzliche Zeit in Anspruch nahm. Durch Domain-Driven Design (DDD) konnten wir die Anwendung in fachlich zusammengehörige Bereiche, sogenannte Bounded Contexts, wie Patientenverwaltung, Terminmanagement, Dokumentenverwaltung und Behandlungsmanagement aufteilen. Dadurch wurden die Verantwortlichkeiten klarer getrennt und die einzelnen Bereiche konnten übersichtlicher entwickelt und getestet werden. Eine Herausforderung war es, die fachlichen Grenzen sinnvoll festzulegen und unnötige Abhängigkeiten zwischen den Bereichen zu vermeiden. Der Einsatz eines Large Language Models (LLM) hat unseren Entwicklungsprozess unterstützt und teilweise beschleunigt, da wir uns Verbesserungsmöglichkeiten für die Implementierung erarbeiten konnten. Besonders bei der Fehlersuche, beim Refactoring und beim Verständnis unbekannter Java-Techniken war diese Unterstützung hilfreich. Allerdings können LLMs auch fehlerhaften, unvollständigen oder für die bestehende Projektarchitektur ungeeigneten Code erzeugen. Deshalb haben wir die Vorschläge kritisch geprüft, bei Bedarf angepasst und die Ergebnisse mithilfe eigener Tests kontrolliert. Um reines Copy-Paste zu vermeiden, haben wir uns die vorgeschlagenen Codeabschnitte erklären lassen, ihre Funktionsweise nachvollzogen und sie selbst in IntelliJ getestet. Besonders gut eignet sich die LLM-Unterstützung für das Erstellen von Testideen, das Erklären von Fehlermeldungen, kleinere Refactorings und Code-Reviews. Weniger geeignet ist sie für komplexe Architekturentscheidungen oder fachliche Entscheidungen, bei denen wichtige Anforderungen nicht eindeutig beschrieben sind. Insgesamt haben wir gelernt, dass TDD, DDD und LLM-Unterstützung die Qualität und Struktur der Softwareentwicklung verbessern können, aber auch eigene Kenntnisse, kritisches Hinterfragen und sorgfältige Überprüfung erfordern. Die Verantwortung für den Code und die getroffenen Entscheidungen bleibt letztlich bei uns als Entwicklerteam.